

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**УЕБ СИСТЕМА ЗА НАБЛЮДЕНИЕ НА
ПОКАЗАТЕЛИ В РЕАЛНО ВРЕМЕ: НТТР И
WEBSOCKET СЪРВЪР НА C++ И КЛИЕНТ БЕЗ
ПРИЛОЖНА РАМКА**

КУРСОВ ПРОЕКТ

по дисциплина „Програмиране за Интернет“

Разработил:

Алекс Цветанов, фак. № 121326006

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Огнян Наков

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА	2
1.1. Развитие на обмена между браузър и сървър	2
1.2. Изместени технологии и техните наследници	2
1.3. Направените избори и цената им	3
II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ	4
2.1. Обща структура и модел на изпълнение	4
2.2. Договор между сървъра и клиента	4
2.3. Свързване на данни на клиента	5
2.4. Протоколи и границите на доверието	5
2.5. Двата формата и събирачът на показатели	6
2.6. Цикъл на сървъра	7
2.7. Клиентска част и проверки	7
III. ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА И РЕЗУЛТАТИ	8
3.1. Методика	8
3.2. Закъснение по трите канала	9
3.3. Сравнение на двата формата за обмен	9
3.4. Поведение при нарастващ брой клиенти	10
ЗАКЛЮЧЕНИЕ	11
ЛИТЕРАТУРА	12
ПРИЛОЖЕНИЕ	13

УВОД

Наблюдението на работеща система е задача, при която закъснението има цена. Ако показателят стигне до оператора със закъснение от няколко секунди, картината на екрана описва състояние, което вече не съществува. Класическият уеб модел, при който браузърът пита сървъра през равни интервали и получава пълна страница в отговор, харчи заявки, когато нищо не се е променило, и въпреки това изостава, когато промяната настъпи между две запитвания. Двупосочната връзка, при която сървърът изпраща стойност в момента, в който я узнае, решава и двата проблема наведнъж.

Настоящият курсов проект разработва **Pulse**: уеб система за наблюдение на показатели в реално време. Сървърната част е програма на C++20, която сама обслужва HTTP/1.1 [1] за първоначалното зареждане и WebSocket [2] за потока от стойности след това. Клиентската част е страница без приложна рамка, която стъпва само на браузърните стандарти: обектния модел на документа [3], модела на събитията, структурната графика във формат SVG [4] и разположението с CSS Grid [5].

Целта е работеща система за наблюдение в реално време, при която всяка тема от учебното съдържание на дисциплината е представена от действащ елемент на системата, а не от отделен демонстрационен пример. Задачите са: анализ на технологиите за обмен между браузър и сървър и обосновка на избора между тях; проектиране на архитектурата и на договора между двете страни; програмна реализация на сървъра на C++20 и на клиента без рамка; и измерване на закъснението, на обема на съобщението и на цената на двата формата за обмен, с анализ на получените числа.

Обхватът е един процес на сървъра, който събира показатели и ги раздава, и една страница в браузъра, която ги изобразява. Извън обхвата остават разпределеното съхранение на исторически редове, удостоверяването на потребители и разгръщането в производствена среда.

I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА

1.1. Развитие на обмена между браузър и сървър

Първото поколение уеб приложения генерира целия документ на сървъра и го изпраща наново при всяко действие: състоянието живее в сървъра, браузърът е програма за изобразяване, а всяка промяна струва пълен обмен. Второто поколение измества част от състоянието в браузъра. Скриптовият език [6] получава достъп до дървото на документа [3] и го променя след зареждането, а асинхронната заявка от скрипт, формализирана по-късно в общ модел за извличане [7], позволява данни да се вземат, без страницата да се разрушава.

Оттук нататък остава един въпрос: кой започва обмена. Периодичното запитване (*polling*) пита през равни интервали и плаща компромис между излишни заявки при кратък интервал и закъснение при дълъг; задържаното запитване намалява едното за сметка на сложност в сървъра, но остава в модела „заявка, отговор“ на HTTP [8]. Съвременните механизми за поток от сървъра са два: събитията, изпращани от сървъра [9], които са еднопосочни, и протоколът WebSocket [2], който започва като заявка за надграждане и превръща вече установената връзка в двупосочен канал за съобщения.

За структурирания обмен съществуват две широко приети представяния. Разширяемият език за разметка [10] носи схема, пространства от имена и утвърден инструментариум за проверка, а нотацията за обекти [11] е по-кратка и се разчита от браузъра без библиотека. Проектът реализира и двата за един и същ поток по три предварително фиксирани показателя: обем по мрежата, време за изписване и време за разбор до пълно дърво.

1.2. Изместени технологии и техните наследници

Част от терминологията в учебното съдържание идва от периода, в който разширяването на браузъра ставаше чрез двоични компоненти. ActiveX изпълнява native код в контекста на страницата и решава реален проблем за времето си, защото тогавашният браузър няма нито графика извън растерни изображения, нито мрежов достъп извън заявката за документ. Изместена е по три причини: обвързаност с една платформа, модел на сигурността, който дава на страницата правата на потребителя, и покриването на същите нужди от стандартите. Проектът третира тези технологии исторически и реализира всяка

тема със стандартизирания ѝ наследник (Табл. 1).

Таблица 1. Съответствие между технологиите от учебната програма и реализираните в проекта

Тема	Реализирано с	Основание
Двоични компоненти (ActiveX)	стандартни браузърни интерфейси	еднаква работа във всеки браузър
Генериране на страници на сървъра (ASP)	HTTP отговор от сървъра на C++20	същата роля, без една платформа
Филтри и преходи като собствено разширение	каскадни стилови преходи и филтри	стандартизирани, хардуерно ускорени
Растерна графика за диаграми	структурна графика SVG [4]	всеки елемент е възел и приема събития
Свързване на данни чрез компонент	собствен слой върху обектния модел [3]	темата остава видима в изходния текст

1.3. Направените избори и цената им

Транспорт. Избран е WebSocket, защото задачата изисква и двете посоки: сървърът изпраща стойности, клиентът изпраща команди. Реализирани са обаче и трите подхода и клиентът превключва между тях по време на работа, защото иначе сравнението в Раздел III щеше да съпоставя измерена стойност с цитирана.

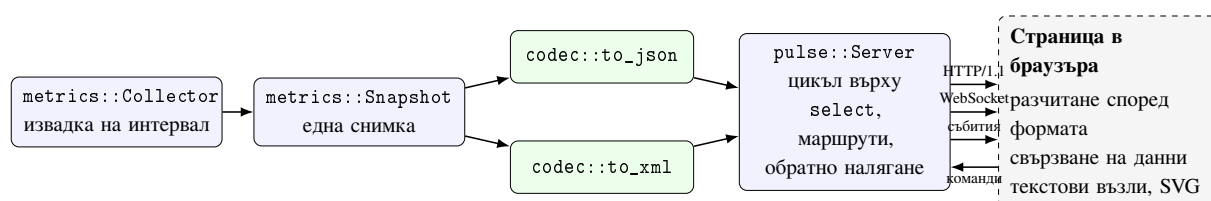
Клиент без приложна рамка. Мотивът е учебен: съдържанието на дисциплината за клиентската част при използване на рамка се превръща в извикване на нейни функции и остава скрито. Без рамка обектният модел, събитията, свързването на данни и структурната графика са видими в изходния текст. Цената е ръчно написан слой за свързване.

Език и зависимости. Сървърът е на C++20 [12] заради прякото управление на паметта и предвидимото време за реакция, което е съществено, когато измерваната величина е закъснението на самата система. Готов сървър на приложения е отхвърлен, защото обработката на HTTP/1.1 [1] и на ръкостискането [2] е част от учебното съдържание, а не спомагателна работа. Хранилището няма задължителна външна зависимост, за да може чужд човек да го изгради на чиста машина от първия път. Цената е, че някои части са по-прости от промишлените си съответствия: анализаторът на разширяемия език за разметка [10] покрива подмножеството, което проектът излъчва.

II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ

2.1. Обща структура и модел на изпълнение

Сървърният процес чете показатели от операционната система, оформя ги в стойностна снимка, превръща снимката в съобщение и го изпраща по изборния от клиента канал. Страницата в браузъра приема съобщението, разчита го и обновява само засегнатите части от документа (Фиг. 1). Сървърът е разделен на шест единици за превод: `net`, `http`, `websocket`, `metrics`, `codec` и `server`. Разделянето следва посоката на данните, а не слоевете на протокола.



Фигура 1. Обща структура на системата и посока на данните

Носещото проектно решение е, че *сериализаторът е отделен от източника на показатели*. Един и същ поток се излъчва в разширяемия език за разметка [10] или в нотацията за обекти [11], без нито един ред от източника или от цикъла да се променя. Без това разделяне сравнението между двата формата щеше да сравнява две реализации, а не два формата.

Сървърът е *еднонишков*. Един цикъл наблюдава слушащия сокет и всички връзки с `select`, обслужва готовите и на всеки интервал взема нова извадка и я разпраща. Няма работни нишки и няма ключалки, защото всяко споделено състояние се пипа от точно една нишка. Цената е заявена предварително: бавен обработчик би спрял всички клиенти, което е приемливо, защото нито един обработчик не прави друго освен да оформи низ от стойности, които вече са в паметта. Всяка връзка носи собствени буфери и режим: HTTP, надградена по WebSocket или отворен поток от събития. Режимът се сменя в средата на буфера.

2.2. Договор между сървъра и клиента

От сървъра идват три вида съобщения: стойностна снимка, потвърждение на команда и съобщение за грешка. Снимката носи пореден номер, времеви отпечатък,

натоварване на процесора, заета памет, броячи на заявки и на връзки, три квантила и хистограма от девет коша. От клиента идват три команди: `format` със стойност `json` или `xml`, `subscribe` и `unsubscribe`, и `interval` между 50 и 5000 милисекунди. Командите вървят само по `WebSocket`, защото другите два транспорта са еднопосочни. Отпечатъкът е нужен само за измерването: клиентът го изважда от собствения си часовник и получава възрастта на стойността при пристигане.

Едно и също съдържание се раздава по три начина: `/ws` отговаря с 101 или 426, `/api/stream` с парчетно кодиран поток от събития, а `/api/metrics.json` и `/api/metrics.xml` с последната снимка за периодично запитване. Отговорът 426 не е грешка в клиента, а заявка, която сървърът не може да изпълни както е написана; стандартът има точно това състояние за случая [8].

2.3. Свързване на данни на клиента

Свързването е явен списък от тройки: път до стойност в снимката, възел от документа и функция, която записва стойността в този възел. При пристигане на снимка се обхождат само тройките с променена стойност, защото при няколко обновявания в секунда пълното преизчисляване на изгледа прави повече работа, отколкото има променени стойности. Диаграмите се изграждат като структурна графика [4]: всяка точка от реда е възел в документа и приема обработчик на събитие и стилев преход, а разположението е зададено с `CSS Grid` [5]. Обработката на събития е съсредоточена чрез *делегиране* върху общия родителски възел, защото възлите на диаграмата се създават и унищожават при всяко обновяване.

Проектът приема четири ограничения, заявени преди да са измерени резултати: състоянието се пази в паметта на процеса и историята се губи при рестарт; поддържа се един източник на показатели; няма удостоверяване, което е приемливо само защото системата се изпълнява локално; интервалът на публикуване е общ за всички клиенти.

2.4. Протоколи и границите на доверието

Изходният текст е разделен на публични договори в `include/`, реализация в `src/`, клиентска част в `web/` и проверки в `tests/`; обемът по файлове е в Приложението. `pulse::net` събира на едно място разликите между `Winsock` и сокетите от семейството `Unix`, така че останалата част от проекта не съдържа нито едно условно компилиране

по операционна система. Там `FD_SETSIZE` се задава на 1024 *преди* включването на `winsock2.h`, защото типът `fd_set` се оразмерява от този макрос при компилация, а стойността по подразбиране от 64 не стига за опита с нарастващ брой връзки.

`http::parse_request` връща едно от три състояния: непълно, разчетено или повредено. Анализаторът не блокира, а съобщава, че наличните байтове не стигат, и цикълът го извиква отново. При успех връща и броя изядени байтове, откъдето идват наведнъж постоянната връзка и приемането на няколко заявки в един пакет.

Кадровият анализатор по WebSocket е мястото, където проектът е най-чувствителен към грешки, защото работи изцяло с данни от мрежата. Отхвърлят се запазен бит без договорено разширение, непознат код на операция, разделен или прекалено дълъг управляващ кадър, дължина в по-дълга форма, отколкото е нужно, дължина над горната граница и кадър от клиента без маска. Проверката на дължината е *преди* заделянето на памет, и същото важи за тялото на HTTP заявка: иначе размерът на заделянето се избира от насрещния.

2.5. Двата формата и събирачът на показатели

Числата и в двата сериализатора минават през една и съща функция с фиксиран брой знаци след десетичната точка: иначе разликата в обема щеше да отразява форматиране на числа, а не разлика в разметката. Разчитането е реализирано и за двата формата, защото времето за разбор е измервана величина. И двата анализатора изграждат *пълно дърво*, тоест сравнението е между еднакъв вид работа, а това е и работата, която браузърът върши при `JSON.parse` и при `DOMParser`. И двата отказват вход извън очакваното вместо да го отгатват, а дълбочината на влагане е ограничена, за да не може вход от мрежата да вкара рекурсията в защитната страница на стека. Анализаторът на XML [10] покрива само подмножеството, което проектът излъчва, което е заявено ограничение, не пропуск.

`metrics::Collector` чете броячите на процеса от `GetProcessTimes` и `GetProcessMemoryInfo` под Windows, от `getrusage` и `/proc/self/statm` под Linux; където няма поддържано четене, се връща нула: липсваща стойност остава липсваща. `steady_clock` мери продължителности вътре в сървъра, защото стенният часовник може да отскочи назад; `system_clock` дава отпечатъка към браузъра.

2.6. Цикъл на сървъра

Цикълът събира слушащия сокет и всички връзки в два набора за `select`: за четене винаги, за писане само когато има какво да се изпрати. Изходящият буфер има таван и при надхвърлянето му връзката се затваря, за да не може един бавен клиент да изчерпи паметта на процеса.

Едно решение заслужава отделно обяснение. Първоначалната реализация избираше как да обработи получените байтове веднъж: HTTP или WebSocket. Това е грешно и се прояви при първото изпълнение на теста за смяна на формата, защото клиент има право да изпрати първия си кадър в същия пакет като заявката за надграждане, тоест остатъкът от буфера принадлежи на другия протокол. Поправката е цикъл, който продължава, докато има напредък (Листинг 1), а не кръпка в пътя, по който тестът мина.

```
1 void process(Connection& connection) {
2     std::size_t before = 0;
3     do {
4         before = connection.incoming.size();
5         switch (connection.mode) {
6             case Mode::Http:      on_http_bytes(connection);      break;
7             case Mode::WebSocket: on_websocket_bytes(connection); break;
8             case Mode::EventStream: connection.incoming.clear();  break;
9         }
10    } while (!connection.incoming.empty() &&
11            connection.incoming.size() != before &&
12            !connection.close_after_write);
13 }
```

Листинг 1: Обработка на получените байтове според режима, `src/server.cpp`

Потокът от събития се рамкира с парчетно кодиране [1], а не със затваряне на връзката, защото дължината на тялото не е известна предварително.

2.7. Клиентска част и проверки

Клиентът е един документ, един стилев файл и един скрипт. Двете функции за разчитане връщат един и същ обект, така че нищо след тях не знае кой формат е пристигнал, а времето около извикването се измерва с `performance.now`. Инструментът за изпълнение на тестовете е `tests/check.hpp`, 119 реда, следствие от липсата на зависимости. Наборът съдържа 35 случая и 586 проверки: анализаторът на HTTP срещу разрязан вход, кадровият кодек срещу примерите от спецификацията [2], двата сериализатора, двата анализатора и работещият сървър по трите транспорта.

III. ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА И РЕЗУЛТАТИ

3.1. Методика

Проверяват се три твърдения, записани *преди* измерването, за да остане една погрешна прогноза видима, вместо да бъде пренаписана след резултата: че двупосочният канал доставя стойност по-бързо от периодичното запитване при еднаква честота; че двата формата се различават по обем и по време за разбор; и че съществува граница по брой едновременни клиенти.

Закъснението се мери като *възраст на стойността при пристигане*: съобщението носи отпечатък от сървъра, а клиентът го изважда от собствения си часовник. Източникът на време от двете страни е `std::chrono::system_clock`; степенен, а не монотонен часовник, защото двата края са различни обекти. Разделителната способност е измерена, а не приета наум: най-малката наблюдавана ненулева стъпка между две четения е 100 ns. Обемът се мери като действително доставени байтове, заглавните части и рамките включително, защото именно доставеното се плаща.

Натоварването се създава от `pulse-demo`, който стартира сървъра в отделна нишка в същия процес и се свързва по локалния интерфейс като обикновен клиент. Браузърът не участва в числата, защото планировчикът на страницата би се смесил с измерваната величина. Оттук следва, че закъсненията не съдържат мрежово време, а показанието за време на процесора включва и работата на измерващия клиент.

Условията са фиксирани предварително и не са променяни, след като са видени резултати. Интервалът на публикуване е 100 ms. Изписването и разборът се измерват в 5 кръга по 10 000 повторения; отчита се средното на *средния* кръг, а най-ниският и най-високият дават мярка за разсейване, защото един кръг се различава от друг с множител, близък до две. За всеки транспорт се събират 65 съобщения, от които първите 5 се отхвърлят. Средата е AMD Ryzen 5 3600, Windows 11 Pro N 10.0.26200, g++ 15.2.0 (MinGW-w64), `CMAKE_BUILD_TYPE=Release`, обмен по 127.0.0.1.

3.2. Закъснение по трите канала

Таблица 2. Възраст на стойността при пристигане и обем на едно обновяване, по вид на канала

Канал	Брой	Средно ms	Медиана ms	95-и проц. ms	Байта на обн.
WebSocket	60	0,47	0,15	0,76	373,7
Събития от сървъра	60	0,66	0,16	3,02	410,7
Периодично запитване	60	42,20	46,59	92,00	561,3

Прогнозата се потвърждава, но не по причината, която би се очаквала (Табл. 2).

Разликата не е в скоростта на преноса: медианите на двата потокови канала са 0,15 и 0,16 ms, тоест неразличими. При периодично запитване стойността вече е *остаряла*, когато клиентът я поиска: сървърът взема извадка на всеки 100 ms, тоест запитване в произволен момент между две извадки получава стойност на средна възраст половин интервал. Измерените 42,20 ms средно съответстват точно на това очакване, а 92,00 ms на деветдесет и петия процентил са близо до пълния интервал. Заключение е по-точно от първоначалното твърдение: периодичното запитване не е бавно, то е *ненавременно*, а съкращаването на интервала не решава проблема, защото всяко запитване струва пълен обмен от заглавни части: 561,3 байта срещу 373,7. Двата потокови канала се различават само по деветдесет и петия процентил, 0,76 срещу 3,02 ms, заради рамкирането: събитието носи рамка от текстови редове и парчетно кодиране [1], а кадърът по WebSocket носи заглавна част от два до четири байта.

3.3. Сравнение на двата формата за обмен

Таблица 3. Обем, време за изписване и време за разбор по формат на обмена

Формат	Байта	Изпис. μs	[мин, макс] μs	Разбор μs	[мин, макс] μs	Възли
JSON	386	6,803	[5,35; 7,45]	17,268	[14,89; 19,73]	42
XML	566	7,522	[7,42; 8,03]	8,191	[7,42; 9,20]	33

Твърдението за обема се потвърждава (Табл. 3): XML е с 46,6% по-обемен, а разликата идва изцяло от разметката.

Твърдението за времето за разбор се потвърждава по посока, обратна на очакваната. XML се разчита за 8,191 μ s срещу 17,268 μ s за JSON. Причината не е в синтаксиса, а в формата на документа: хистограмата се записва в JSON като масив от девет обекта с по две полета, което дава 42 възела, а в XML като девет елемента с признак и текстово съдържание, което дава 33. Изграждането на възел е основната цена и при двата анализатора. Затова резултатът не бива да се обобщава до твърдението, че XML се разчита по-бързо от JSON. Правилното заключение е по-тясно: *цената на разбора се определя главно от броя на изградените възли, а той зависи от това как е моделиран документът, а не от избрания синтаксис*. Времето за изписване е практически еднакво, 6,803 срещу 7,522 μ s, а разсейването между кръговете при JSON е от порядъка на самата разлика. Компресията не е измервана.

3.4. Поведение при нарастващ брой клиенти

Таблица 4. Доставени съобщения и закъснение при нарастващ брой едновременни връзки

Клиенти	Доставени	Очаквани	Запазени %	Средно ms	95-и проц. ms
1	30	30	100,0	0,30	0,46
8	240	240	100,0	0,29	0,55
32	960	960	100,0	0,87	2,09
128	3 840	3 840	100,0	2,04	4,89

Границата не е достигната (Табл. 4). До 128 едновременни връзки не е изпуснато нито едно съобщение, а закъснението нараства от 0,30 до 2,04 ms, тоест остава далеч под интервала на публикуване, защото при 128 връзки един цикъл оформя 128 кадъра, преди да се върне към чакане. Понеже доставените съвпадат с очакваните, измереното е поведение на сървър, а не на инструмента. Границата не е потърсена по-нагоре, защото измерваният клиент дели машина със сървър.

От трите предварителни твърдения първите две са потвърдени, а третото остава **непроверено**. **Ограничения на самото измерване:** закъсненията са долна граница за реална мрежа, компресията не е измервана, а опитът е изпълнен веднъж, тоест отчетеното разсейване е вътре в едно изпълнение.

ЗАКЛЮЧЕНИЕ

Разработена е система за наблюдение на показатели в реално време: сървър на C++20 [12], който сам обслужва HTTP/1.1 [1] и WebSocket [2], и клиент в брауъра без приложна рамка, стъпил пряко върху обектния модел на документа [3], модела на събитията, структурната графика [4] и CSS Grid [5]. Обменът е реализиран в два формата [10, 11], за да бъде разликата между тях измерена.

Ограничения. Слой за свързване на данни е написан ръчно и изисква явно описание на всяка зависимост, което при разрастване на интерфейса става източник на пропуски. Състоянието се пази в паметта на процеса и се губи при рестарт. Няма удостоверяване на потребители. Границата по брой едновременни клиенти не е намерена. Тези ограничения са заявени преди измерването, а не открити след него.

Инженерна трактовка. Измерването дава два извода, които се пренасят извън този проект. Първият: периодичното запитване не изостава защото пренася бавно, а защото връща стойност, която вече е остаряла, тоест съкращаването на интервала не е решение, а размяна на едно затруднение за друго, защото всяко запитване струва пълен обмен от заглавни части. Между двата потокови канала разликата е малка и WebSocket се откроява само по по-стегнатото рамкиране и по това, че носи и обратната посока. Вторият извод е обратен на очакваното: разширяемият език за разметка се оказва с 46,6% по-обемен, но се разчете за по-малко от половината от времето на нотацията за обекти, защото дървото му съдържа по-малко възли за същите данни. Цената на разбора следва *броя на изградените възли*, тоест начина, по който е моделиран документът, а не избрания синтаксис. При проектиране на обмен това означава, че формата на съобщението заслужава повече внимание от спора кой синтаксис да се ползва.

И двата извода опровергават формулировка, записана преди опита. Ако твърденията не бяха записани предварително, разликата между очакваното и измереното нямаше да остане видима.

Насоки за по-нататъшна работа. Съхранение на исторически редове извън паметта на процеса, обединяване на няколко източника, удостоверяване на потребители и разширяване на слоя за свързване с автоматично проследяване на зависимостите.

ЛІТЕРАТУРА

- [1] R. Fielding, M. Nottingham, and J. Reschke, “HTTP/1.1,” Request for Comments RFC 9112, Internet Engineering Task Force, June 2022.
- [2] I. Fette and A. Melnikov, “The WebSocket protocol,” Request for Comments RFC 6455, Internet Engineering Task Force, Dec. 2011.
- [3] WHATWG, “DOM standard.” Living Standard.
- [4] W3C SVG Working Group, “Scalable vector graphics (SVG) 2.” W3C Specification.
- [5] W3C CSS Working Group, “CSS grid layout module level 1.” W3C Specification.
- [6] Ecma International, “ECMA-262: ECMAScript language specification.” Standard.
- [7] WHATWG, “Fetch standard.” Living Standard.
- [8] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” Request for Comments RFC 9110, Internet Engineering Task Force, June 2022.
- [9] WHATWG, “HTML standard.” Living Standard.
- [10] W3C XML Core Working Group, “Extensible markup language (XML) 1.0 (fifth edition).” W3C Recommendation, Nov. 2008.
- [11] T. Bray, “The JavaScript object notation (JSON) data interchange format,” Request for Comments RFC 8259, Internet Engineering Task Force, Dec. 2017.
- [12] ISO/IEC, “ISO/IEC 14882:2020, programming languages, C++.” International Standard, 2020.

ПРИЛОЖЕНИЕ

Пълният изходен текст, тестовете и суровият отчет от измерването са в хранилището на проекта: <https://github.com/Alex-Tsvetanov/pulse>. Отчетът, от който произхожда всяко число в Раздел III, е файлът `measurements/report.txt`, 46 реда, изведен без редакция от `pulse-demo --bench`. Разпределението на изходния текст по файлове е дадено в (Табл. 5).

Таблица 5. Дърво на хранилището

Файл	Редове	Съдържание
<code>include/pulse/*.hpp</code>	478	шестте публични договора
<code>src/net.cpp</code>	179	сокети: Winsock и Unix зад един интерфейс
<code>src/http.cpp</code>	263	разбор на заявка, отговор, видове съдържание
<code>src/websocket.cpp</code>	270	SHA-1, Base64, кадри, събиране на части
<code>src/metrics.cpp</code>	162	бройчи на процеса и хистограма
<code>src/codec.cpp</code>	628	двата формата, изписване и разчитане
<code>src/server.cpp</code>	503	цикъл на събитията, маршрути, транспорти
<code>src/main.cpp</code>	83	двоичният файл на сървъра
<code>src/demo.cpp</code>	537	демонстрация и измервателен инструмент
<code>tests/check.hpp</code>	119	инструмент за изпълнение на тестове
<code>tests/tests.cpp</code>	720	35 случая, 586 проверки
<code>web/index.html</code>	109	един документ
<code>web/app.css</code>	316	разположение, теми, преходи
<code>web/app.js</code>	345	транспорти, свързване, диаграми

Екранни снимки на интерфейса не са включени в отчета. Интерфейсът се вижда, като се стартира `pulse-server` и се отвори `http://localhost:8080`.